福州大学

《Linux 操作系统设计实践》实验报告

学号：102401339　姓名：任宝硕　专业：计算机科学与技术　班级：3 班

实验二　进程通信

实验环境

本实验在 macOS 27.0（Darwin 27.0.0，arm64）环境下完成，使用 Apple clang 21.0 编译器辅以 GNU Make 3.81 通过在 Otty 终端上运行的 zsh 编译运行。

实验内容

编写了一个客户端与服务器端通信程序，由 server.c、client.c、shm.h 和 Makefile 四个文件组成。server.c 负责创建共享内存、发送消息和接收回复；client.c 负责读取服务器消息并写入回复；shm.h 定义双方使用的键值 SHM_KEY 和结构体 shm_data。Makefile 将 server.c 和 client.c 编译为 server 和 client，并提供 clean 目标清理可执行文件。

共享内存的键值为 1234，大小为 sizeof(struct shm_data)。结构体中有两个整型标志 server_ready 和 client_ready，分别表示服务器消息和客户端回复已写入；server_msg 和 client_msg 是两个长度为 128 的字符数组，用于存放消息。

服务器启动后调用 shmget()，参数 IPC_CREAT|0666 表示共享内存段不存在时创建，并设置读写权限。随后调用 shmat()，将共享内存附加到进程地址空间。shmget()或 shmat()失败时，程序通过 perror()输出错误信息，并以退出码 1 结束。附加成功后，服务器调用 memset()清零结构体，将“Hello from server!”写入 server_msg，再把 server_ready 置为 1，等待客户端回复。

客户端使用相同的键值调用 shmget()，但不传入 IPC_CREAT。若共享内存段不存在，客户端输出错误信息，提示先运行./server，并以退出码 1 结束。获取共享内存后，客户端调用 shmat()，等待 server_ready 变为非零值，再输出 server_msg 中的消息。随后将“Hi, this

is client.”写入 client_msg，把 client_ready 置为 1，调用 shmdt()分离共享内存并退出。

服务器等待 client_ready 变为非零值，输出 client_msg 中的回复，再调用 shmdt()分离共享内存，通过 shmctl(id, IPC_RMID, NULL)标记删除该段。双方等待消息时都采用轮询，标志为 0 就调用 sleep(1)，休眠后再次检查。。

程序代码

server.c

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include "shm.h"

int main(void) {
  int id = shmget(SHM_KEY, sizeof(struct shm_data), IPC_CREAT | 0666);
  if (id < 0) {
    perror("shmget");
    return 1;
  }

  struct shm_data *p = shmat(id, NULL, 0);
  if (p == (void *)-1) {
    perror("shmat");
    return 1;
  }
  memset(p, 0, sizeof(struct shm_data));

  strcpy(p->server_msg, "Hello from server!");
  p->server_ready = 1;
  printf("[服务器] 已写入消息，等待客户端...\n");

  /* 等待客户端回复 */
  while (!p->client_ready) {
    sleep(1);
  }

  printf("[服务器] 收到客户端回复：%s\n", p->client_msg);

  shmdt(p);
  shmctl(id, IPC_RMID, NULL); /* 删除共享内存 */
```

```
    printf("[服务器] 共享内存已删除，退出\n");
    return 0;
  }
```

client.c

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>

#include "shm.h"

int main(void) {
  int id = shmget(SHM_KEY, sizeof(struct shm_data), 0666);
  if (id < 0) {
    perror("shmget（请先运行 ./server）");
    return 1;
  }

  struct shm_data *p = shmat(id, NULL, 0);
  if (p == (void *)-1) {
    perror("shmat");
    return 1;
  }

  /* 等待服务器消息 */
  while (!p->server_ready) {
    sleep(1);
  }

  printf("[客户端] 收到服务器消息：%s\n", p->server_msg);

  strcpy(p->client_msg, "Hi, this is client.");
  p->client_ready = 1;
  printf("[客户端] 已写入回复，退出\n");

  shmdt(p);
  return 0;
}
```

shm.h

```h
#ifndef SHM_H
#define SHM_H

#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_KEY 1234

struct shm_data {
  int server_ready; /* 服务器已写入消息 */
  int client_ready; /* 客户端已写入回复 */
  char server_msg[128];
  char client_msg[128];
};

#endif
```

child.c

```c
#include "child.h"

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include <sys/wait.h>

/* fork 子进程，子进程 exec 加载 hello，父进程等待 */
void create_child(void) {
  pid_t pid = fork();
  if (pid < 0) {
    perror("fork");
    exit(1);
  }

  if (pid == 0) {
    execl("./hello", "hello", NULL);
    perror("exec");
    exit(1);
  }

  wait(NULL);
  printf("子进程 %d 已结束\n", pid);
}
```

Makefile

```
all: main hello

main: main.o child.o
    gcc -o main main.o child.o

hello: hello.c
    gcc -o hello hello.c

clean:
    rm -f main hello *.o

.PHONY: all clean
```

运行截图



截图中，左侧终端运行服务器，右侧终端运行客户端。服务器写入消息后等待回复，客户端读取并输出“Hello from server!”，再写入回复“Hi, this is client.”。服务器收到回复后输出共享内存删除提示，双方退出，完成一次消息往返。

实验总结

1. 实验程序由 server 和 client 两个进程完成一次消息往返。双方使用 shm.h 中的键值和结构体布局访问同一段共享内存。服务器先写入消息，客户端读取后写入回复，服务器收到回复后负责清理共享内存。
2. 实验中，我学习了 shmget()和 shmat()的用法。shmget()返回共享内存段的标识符，shmat()返回附加后的地址，程序通过结构体指针读写其中的数据。两者的失败返回值不同：shmget()返回-1，shmat()返回(void*)-1，不能用 NULL 判断 shmat()是否失败。
3. shmget()的第三个参数包含创建标志和权限位。服务器使用 IPC_CREAT|0666，客户端使用 0666。IPC_CREAT 表示段不存在时创建，0666 是八进制权限值，表示所有者、同组用户和其他用户都有读写权限。如果键值对应的段已存在，服务器会获取已有段，

再调用 memset()清零。先运行客户端是否报错，取决于该段是否存在；若上次运行留下了共享内存，客户端可能读到旧消息。

4. 共享内存供两个进程访问同一段数据，程序还需要安排读写顺序。本程序用 server_ready 和 client_ready 轮询消息状态，sleep(1)避免等待时反复空转，也会带来等待延迟。普通整型标志没有提供同步保证，sleep()也不能替代同步机制。若要保证进程间的读写顺序，需要配合信号量等机制。当前代码没有设置超时，对方未写入消息时会一直等待。
5. shmdt()分离当前进程附加的共享内存，IPC_RMID 标记删除共享内存段，所有进程分离后才会释放。客户端只负责分离，服务器负责标记删除。如果服务器中途退出，可能留下共享内存段，可用 ipcs -m 查看，再用 ipcrm -m shmid 删除对应的段。当前代码没有检查 shmdt()和 shmctl()的返回值，因此服务器输出“共享内存已删除”不能用来确认删除操作成功。